

We cannot load the datasets into GitHub because they are larger than 100MBs or use the GitLFS. The alternative, work in Kaggle's environment for code and other computation.

In GitHub

Under the repository, create files under Notebooks. Then import all scripts under the respective notebooks.

Data Loading and Pre-processing

Loading and Viewing

First Five Records

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	0.090794
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	-0.166974
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	0.207643
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	-0.054952
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	0.753074

Last Five Records

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9
284802	172786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	-2.606837	-4.918215	7.305334	1.914428
284803	172787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	1.058415	0.024330	0.294869	0.584800
284804	172788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	3.031260	-0.296827	0.708417	0.432454
284805	172788.0	-0.240440	0.530483	0.702510	0.689799	-0.377961	0.623708	-0.686180	0.679145	0.392087
284806	172792.0	-0.533413	-0.189733	0.703337	-0.506271	-0.012546	-0.649617	1.577006	-0.414650	0.486180

Dataset Information

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column  Non-Null Count  Dtype  
---  -
0    Time    284807 non-null    float64
1    V1       284807 non-null    float64
2    V2       284807 non-null    float64
3    V3       284807 non-null    float64
4    V4       284807 non-null    float64
5    V5       284807 non-null    float64
6    V6       284807 non-null    float64
7    V7       284807 non-null    float64
8    V8       284807 non-null    float64
9    V9       284807 non-null    float64
10   V10      284807 non-null    float64
11   V11      284807 non-null    float64
12   V12      284807 non-null    float64
13   V13      284807 non-null    float64
14   V14      284807 non-null    float64
15   V15      284807 non-null    float64
16   V16      284807 non-null    float64
17   V17      284807 non-null    float64
18   V18      284807 non-null    float64
19   V19      284807 non-null    float64
20   V20      284807 non-null    float64

```

Descriptive Statistics

```

[6]: # =====
# STEP 6: Descriptive Statistics
# =====

display(df.describe().T)

```

	count	mean	std	min	25%	50%	75%	max
Time	284807.0	9.481386e+04	47488.145955	0.000000	54201.500000	84692.000000	139320.500000	172792.000000
V1	284807.0	1.168375e-15	1.958696	-56.407510	-0.920373	0.018109	1.315642	2.454930
V2	284807.0	3.416908e-16	1.651309	-72.715728	-0.598550	0.065486	0.803724	22.057720
V3	284807.0	-1.379537e-15	1.516255	-48.325589	-0.890365	0.179846	1.027196	9.382550
V4	284807.0	2.074095e-15	1.415869	-5.683171	-0.848640	-0.019847	0.743341	16.875340
V5	284807.0	9.604066e-16	1.380247	-113.743307	-0.691597	-0.054336	0.611926	34.801660
V6	284807.0	1.487313e-15	1.332271	-26.160506	-0.768296	-0.274187	0.398565	73.301620
V7	284807.0	-5.556467e-16	1.237094	-43.557242	-0.554076	0.040103	0.570436	120.589450

Missing Values

```

missing = df.isnull().sum()

missing = pd.DataFrame({
    "Missing Values": missing,
    "Percentage": (missing/len(df))*100
})

display(missing[missing["Missing Values"]>0])

print("Total Missing Values:", df.isnull().sum().sum())

```

Missing Values	Percentage
----------------	------------

Duplicate Records

```

# =====
# STEP 8: Duplicate Records
# =====

duplicates = df.duplicated().sum()

print("Duplicate Records:", duplicates)

```

Duplicate Records: 1081

Classes (Class Imbalance)

```

class_counts = df["Class"].value_counts()

display(class_counts)

fraud_percentage = df["Class"].value_counts(normalize=True)*100

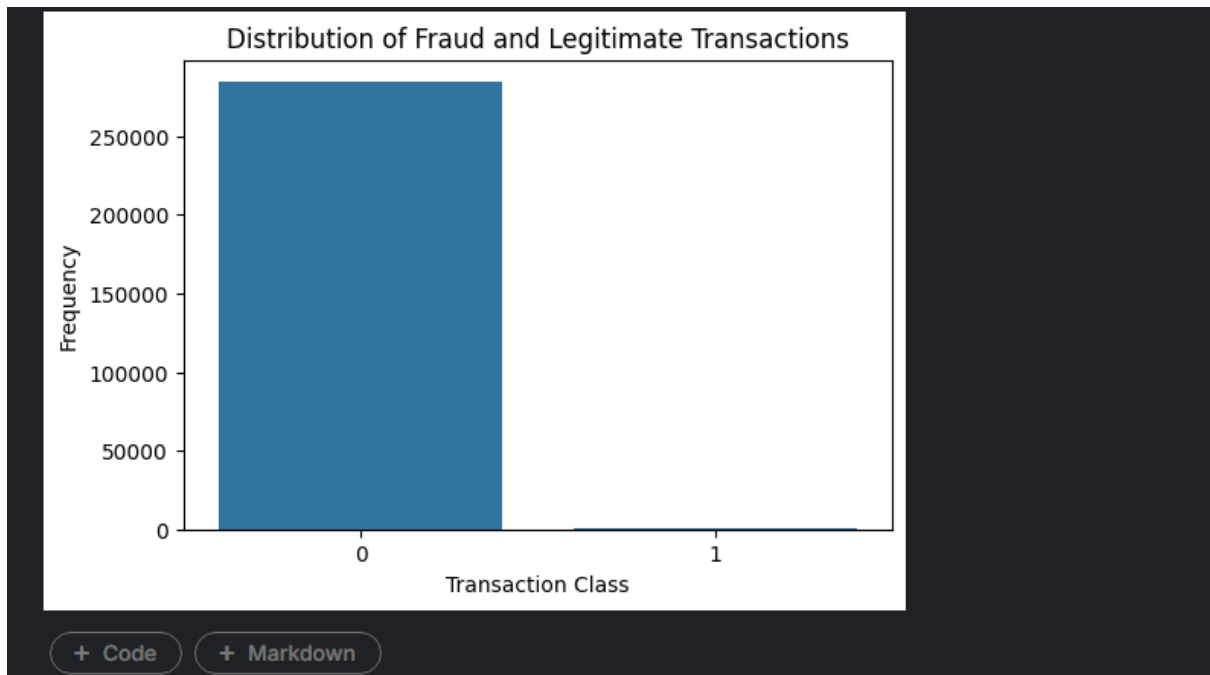
display(fraud_percentage)

```

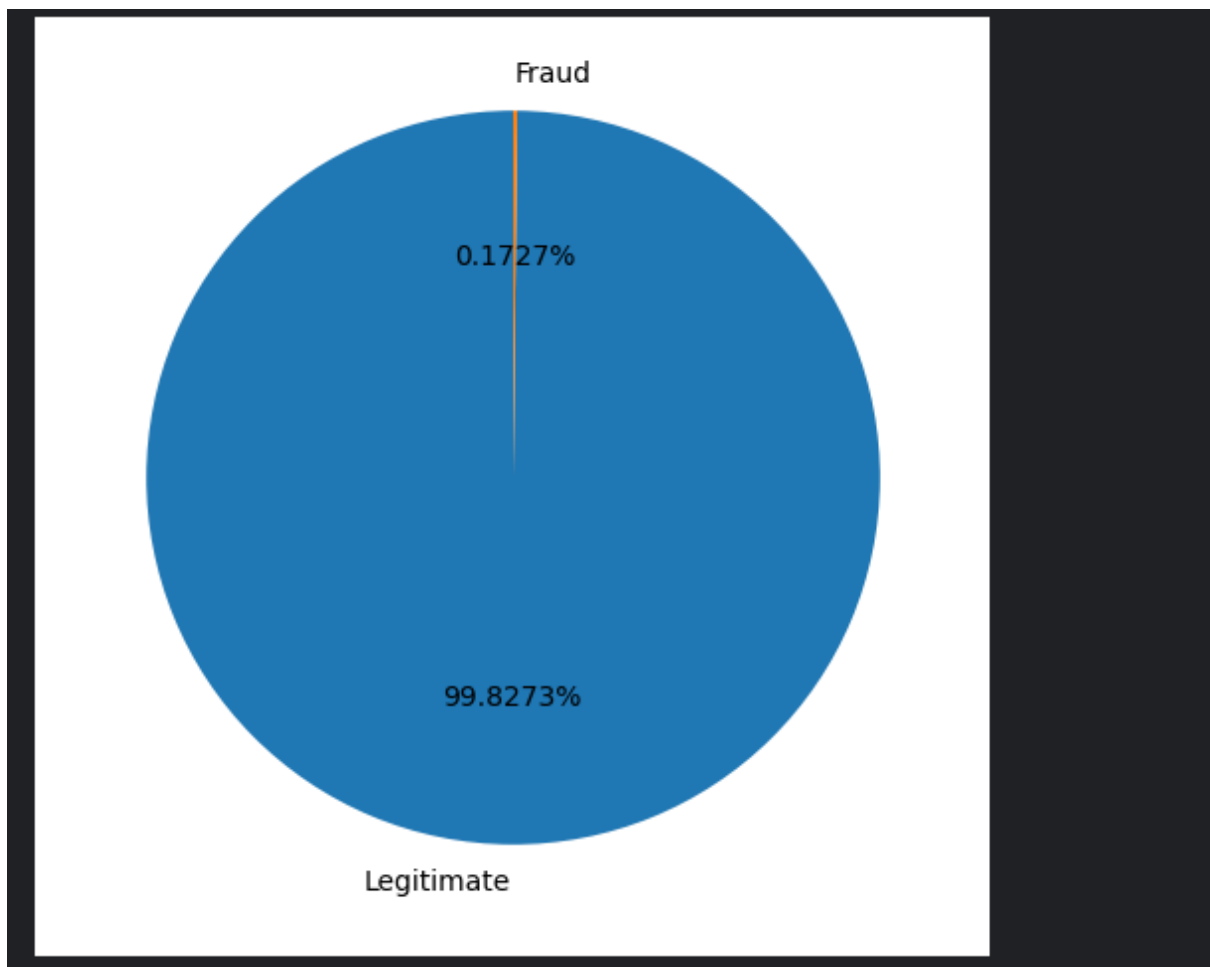
```

Class
0    284315
1       492
Name: count, dtype: int64
Class
0    99.827251
1     0.172749
Name: proportion, dtype: float64

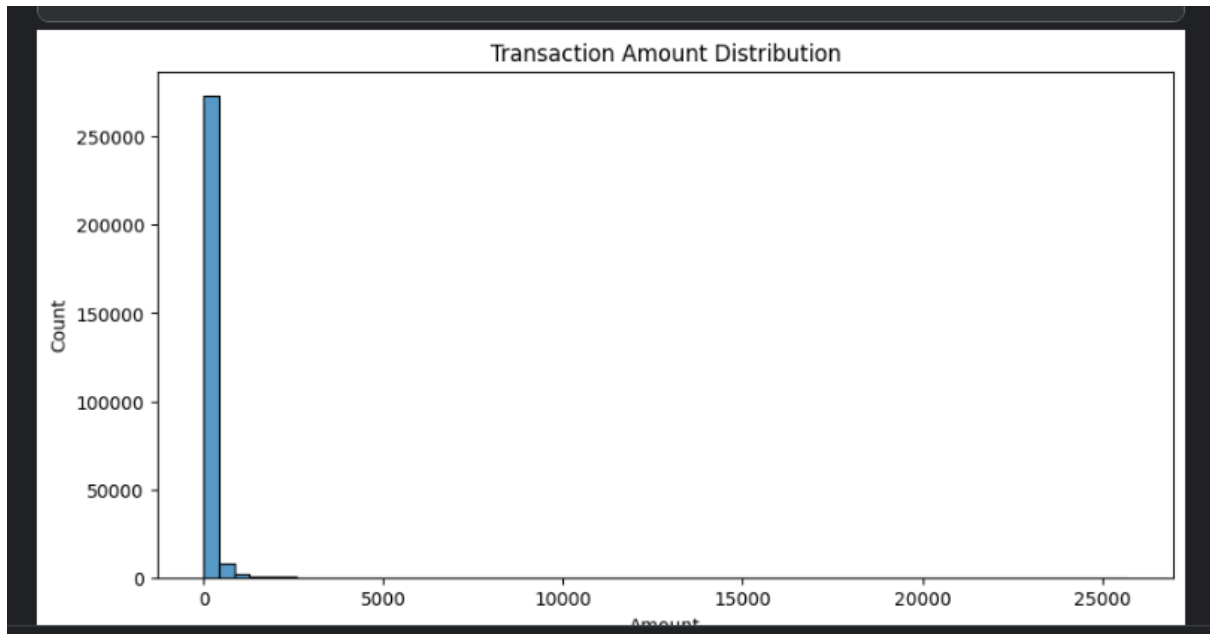
```



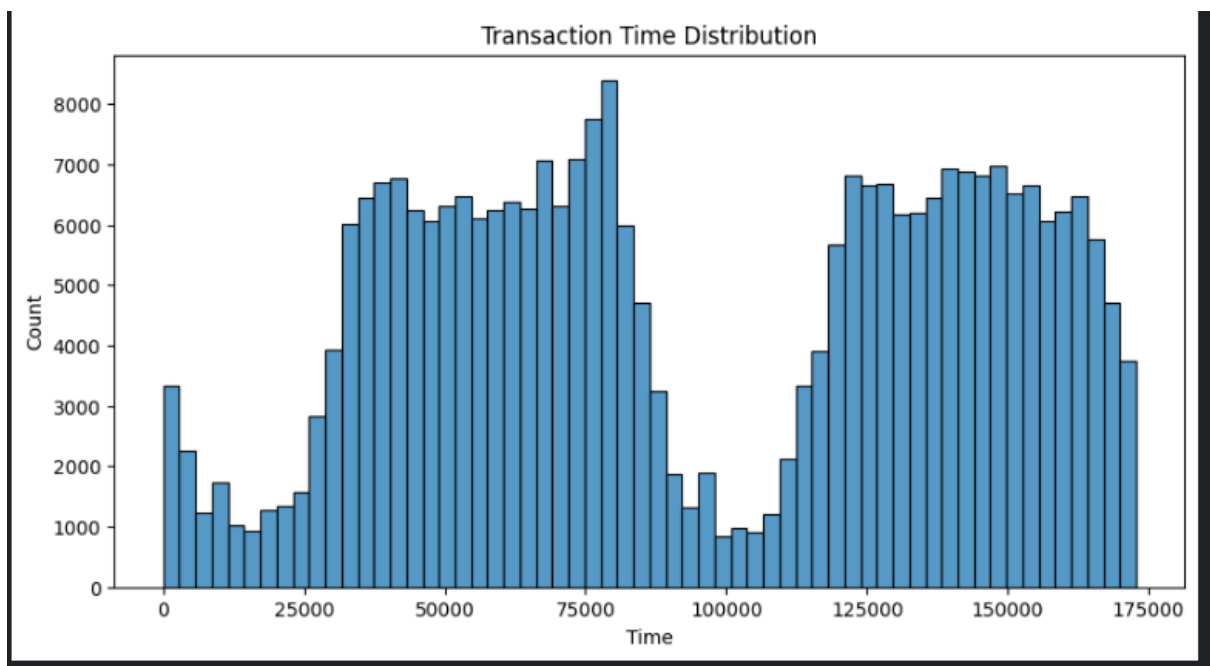
Class Imbalances



Transaction Amount Distribution



Transaction Time Details



Pre-Processing

Removal of Duplicates

```
Original Shape: (284807, 31)
Duplicate Records: 1081
New Shape: (283726, 31)
```

Separation of Features and Targets

Scaling

Smote

```
After SMOTE
Class
0    226602
1    226602
Name: count, dtype: int64
```

Verify Dataset Sizes

```
Original Training Set
(226980, 30)
(226980,)

Balanced Training Set
(453204, 30)
(453204,)

Testing Set
(56746, 30)
(56746,)
```

Pre-processed datasets saved

```
# =====
# STEP 11: Save Processed Data
# =====

X_train_smote.to_csv("X_train.csv", index=False)
X_test.to_csv("X_test.csv", index=False)

y_train_smote.to_csv("y_train.csv", index=False)
y_test.to_csv("y_test.csv", index=False)

print("Processed datasets saved.")
```

Processed datasets saved.

